UNIVERSIDAD DEL VALLE DE GUATEMALA

Colegio Universitario



Proyecto No. 2 — Programación Paralela con MPI (DES por fuerza bruta)

Catedrático: Marlón Fuentes

Jose Ordoñez– 231329

Pablo Méndez – 23975

Osman De León - 23428

CC3069 Computación Paralela y Distribuida

Sección 40

Guatemala, octubre de 2026

Índice

1. Introducción *4*

2. Antecedentes *4*

3. Marco teórico *4*

3.1 Cómo se cifra y descifra **4**

3.2 El criterio para saber si acertamos **4**

3.3 Trabajo en equipo con MPI **5**

3.4 Speedup y eficiencia **5**

3.5 Por qué la llave afecta el resultado **5**

4. Objetivos *5*

4.1 Objetivo general **5**

4.2 Objetivos específicos **5**

5. Cómo funciona el programa *5*

6. Versión secuencial *6*

7. Versión paralela con MPI *6*

7.1 Organización general **6**

7.2 Dos formas de repartir **6**

7.3 Comunicación entre procesos **6**

7.4 Uso y flujo de MPI_Irecv, MPI_Send y MPI_Wait **6**

8. Pruebas y resultados *7*

8.1 Verificación de que el programa funciona **7**

8.2 Tres llaves de distinta dificultad **8**

8.3 Comparación de las dos formas de repartir **8**

8.4 Textos de otros grupos **9**

8.5 Ejemplo de 10 mediciones **9**

9. Conclusiones *10*

10. Recomendaciones *10*

11. Referencias *11*

A. Diagrama de flujo del algoritmo DES *11*

B. Diagrama de flujo del programa *12*

C. Catálogo de funciones *13*

D. Bitácora de pruebas *14*

1. Introducción

En este proyecto se usó un programa que encuentra la llave con la que se cifró un texto. Probar llaves una por una hasta que el texto descifrado contenga una frase conocida. En nuestro caso esa frase es “sudo make me a sandwich”.

Primero usamos una versión secuencial que prueba las llaves en orden. Después la adaptamos para repartir el trabajo entre varios procesos con MPI y así terminar más rápido. También probamos dos formas de repartir las llaves para ver cuál funciona de manera más estable.

Este informe presenta el contexto del cifrado DES, cómo funciona nuestro programa, y las mediciones de tiempo, speedup y eficiencia. Las tablas de resultados quedan marcadas para completarlas con las corridas finales.

2. Antecedentes

DES es un método clásico para cifrar información por bloques. Toma bloques de texto de 8 bytes y los transforma usando una llave de 56 bits a través de 16 rondas de operaciones. Para descifrar se aplica el mismo procedimiento pero en orden inverso.

Como la llave tiene 56 bits, existen más de 72 mil billones de combinaciones posibles (2^56). Probarlas todas es imposible en la práctica, por eso en el proyecto trabajamos con un rango reducido, por ejemplo 2^24 (aproximadamente 16.7 millones de llaves), que sí se puede medir en segundos o minutos.

Nuestro texto de prueba tiene 130 palabras, dentro del máximo de 350, y al cifrarlo ocupa 768 bytes. Además recibimos textos de otros grupos para poner a prueba nuestro programa, cada uno con su propia frase clave.

3. Marco teórico

3.1 Cómo se cifra y descifra

Cifrar es mezclar cada bloque con la llave en varias rondas hasta que el texto queda irreconocible. Descifrar es deshacer esos pasos con la misma llave. Si se usa una llave equivocada, el resultado es texto sin sentido.

3.2 El criterio para saber si acertamos

Como sabemos que el texto original contiene una frase específica, basta con buscar esa frase en cada intento. Si aparece, encontramos la llave. Para eso el programa copia el texto cifrado, lo intenta descifrar con una llave y busca la frase dentro del resultado.

3.3 Trabajo en equipo con MPI

MPI permite que varios procesos colaboren, cada uno con su parte del trabajo. En nuestro programa un proceso coordina y reparte el texto cifrado a los demás, todos buscan en su rango asignado y cuando alguien encuentra la llave avisa al resto para que se detengan. Al final se juntan los tiempos y la cantidad de intentos.

3.4 Speedup y eficiencia

Speedup = tiempo secuencial / tiempo paralelo *Eficiencia = speedup / número de procesos*

El speedup dice cuántas veces más rápido fue el programa paralelo. La eficiencia dice qué tan bien se aprovecharon los procesos. Lo ideal sería que 4 procesos fueran 4 veces más rápidos, pero en la práctica siempre hay costos de coordinación.

3.5 Por qué la llave afecta el resultado

Si las llaves se reparten por bloques seguidos, la suerte influye mucho: si la llave buscada está al inicio del bloque de un proceso, ese proceso la halla rapidísimo y el speedup parece altísimo; si está al final del primer bloque, casi no hay ganancia. Por eso también probamos un reparto intercalado (cíclico), que distribuye mejor esa suerte y suele dar resultados más estables.

4. Objetivos

4.1 Objetivo general

Encontrar llaves DES por fuerza bruta con un programa paralelo en MPI y evaluar cuánto mejora frente a la versión secuencial.

4.2 Objetivos específicos

- Hacer funcionar el cifrado DES y la búsqueda secuencial.
- Repartir la búsqueda con MPI y comparar dos formas de distribuir el trabajo.
- Medir tiempos con 10 repeticiones por caso, variando el número de procesos, y calcular speedup y eficiencia.
- Descifrar al menos 2 textos de otros grupos.

5. Cómo funciona el programa

El equipo desarrolló el código en C: un módulo para cifrar y descifrar con DES, un buscador secuencial, un programa para cifrar archivos de texto y el buscador paralelo con MPI. Además hay scripts que repiten las mediciones y guardan los resultados en archivos CSV.

El uso es simple: se indica el archivo cifrado, la frase a buscar y el límite de llaves. El programa responde con la llave encontrada (o avisa si no está en el rango), el texto descifrado, cuántas llaves probó y cuánto tardó.

6. Versión secuencial

La versión secuencial fue la base de todo. Primero resolvimos que la librería original del ejemplo ya no existe en los sistemas actuales, así que la reemplazamos con OpenSSL, que ofrece el mismo cifrado DES.

También detectamos que la forma en que el ejemplo original construía la llave daba resultados distintos según la máquina, así que dejamos una versión corregida por defecto y la opción de usar la original para comparar. La regla importante es no mezclar textos cifrados con programas de distinta versión.

7. Versión paralela con MPI

7.1 Organización general

Un proceso coordina y también busca. Él lee el archivo y comparte el contenido con todos. Después cada proceso revisa sus llaves por grupos de 256, revisando de vez en cuando si ya hay noticias de los demás, para no detenerse a cada rato.

7.2 Dos formas de repartir

Por bloques: se divide el rango en partes seguidas, una por proceso. Es lo más directo, pero muy sensible a la suerte de dónde cayó la llave.

Cíclica (mejora): se reparten de forma intercalada (proceso 0 prueba 0, 4, 8...; proceso 1 prueba 1, 5, 9... con 4 procesos). Así todos avanzan parejo y los tiempos son más consistentes al agregar procesos.

7.3 Comunicación entre procesos

Cada trabajador avisa una sola vez si halló la llave o si terminó sin éxito. Si alguien la encuentra, el coordinador pide a los demás que se detengan. Al final se recogen el tiempo del proceso más lento y la suma de intentos. El programa también valida los datos de entrada (archivo, frase, límite) para evitar fallos de comunicación.

7.4 Uso y flujo de MPI_Irecv, MPI_Send y MPI_Wait

La idea general del flujo es: cada trabajador avisa una sola vez, el coordinador recoge esos avisos y, en cuanto hay solución, ordena a todos detenerse. Así se ve cada una en ese flujo:

Primitiva	**Qué hace**	**Cómo la usamos en el programa**

MPI_Irecv	Anuncia “estoy listo para recibir” sin quedarse esperando; la revisión se hace después.	El coordinador escucha los reportes de cualquier trabajador y cada trabajador escucha la orden de parada, todo mientras siguen buscando (se revisa entre lotes de 256 llaves).
MPI_Send	Envía un mensaje y espera hasta que el sistema puede completar el envío; eso no garantiza que el otro ya lo haya leído.	Es la forma de avisar del flujo: el trabajador le reporta al coordinador si halló la llave o si terminó, y el coordinador ordena detenerse a los demás. En el código estos avisos usan su versión no bloqueante para no frenar la búsqueda, pero el intercambio es el mismo.
MPI_Wait	Espera hasta que una operación anunciada termina por completo.	Los trabajadores la usan para confirmar su reporte y para recibir la parada final; el coordinador para confirmar que la orden llegó a todos antes de cerrar.

8. Pruebas y resultados

Todas las mediciones se hacen en la misma computadora con 10 repeticiones por cada combinación de texto, número de procesos (2, 4 y 6) y forma de reparto (bloques o cíclica). El límite usado es 2^24.

8.1 Verificación de que el programa funciona

Antes de medir tiempos se corre una prueba automática que verifica que el programa encuentra la llave cuando existe, que avisa cuando no está en el rango y que maneja bien errores como archivo inexistente o parámetros inválidos.

```
OK: encontrada n=1 modo=bloques
OK: no encontrada n=1 modo=bloques
OK: encontrada n=1 modo=ciclica
OK: no encontrada n=1 modo=ciclica
OK: encontrada n=2 modo=bloques
```

```
OK: no encontrada n=2 modo=bloques
OK: encontrada n=2 modo=ciclica
OK: no encontrada n=2 modo=ciclica
OK: encontrada n=4 modo=bloques
OK: no encontrada n=4 modo=bloques
OK: encontrada n=4 modo=ciclica
OK: no encontrada n=4 modo=ciclica
OK: archivo inexistente
OK: palabra vacia
OK: modo invalido
OK: limite invalido
Pruebas defensivas completadas: 16
```

8.2 Tres llaves de distinta dificultad

Para ver el efecto de la suerte usamos tres llaves oficiales adaptadas al rango 2^24: fácil 8388609, media 10485760 y difícil 3687301, todas sobre el mismo texto y frase.

Cuadro 1: Resumen por llave con reparto cíclico.

Llave	**Tiempo secuencial (s)**	**Paralelo P=2**	**Paralelo P=4**	**Paralelo P=6**	**Speedup P=4**	**Eficiencia**
8388609 (facil)	45.260 ±0.251	23.625 ±0.45	12.310 ±0.10	8.844 ±0.11	3.677	91.9 %
10485760 (media)	56.410 ±0.431	29.686 ±0.46	15.526 ±0.19	11.181 ±0.14	3.633	90.8 %
3687301 (dificil)	20.010 ±0.139	10.601 ±0.05	5.439 ±0.02	3.919 ±0.02	3.679	92.0 %

8.3 Comparación de las dos formas de repartir

Cuadros 2a, 2b y 2c: Bloques vs. cíclica por llave.

Procesos	**Bloques (s)**	**Ciclica (s)**	**Speedup bloques**	**Speedup ciclica**
2	0.00152	23.625	29 777 *	1.916
4	0.00275	12.310	16 458 *	3.677
6	0.00313	8.844	14 460 *	5.118

Llave media 10485760

Procesos	Bloques (s)	Ciclica (s)	Speedup bloques	Speedup ciclica
2	11.477462	29.686	4.915	1.900
4	12.310869	15.526	4.582	3.633
6	13.296889	11.181	4.242	5.045

Llave difícil 3687301

Procesos	Bloques (s)	Ciclica (s)	Speedup bloques	Speedup ciclica
2	21.050328	10.601	0.951	1.888
4	21.630015	5.439	0.925	3.679
6	5.660663	3.919	3.535	5.106

8.4 Textos de otros grupos

Probamos los textos texto6 y texto7 y textoequipo10 con sus frases clave, 10 veces cada uno y con 2, 4 y 6 procesos.

Cuadro 3: Textos externos.

Texto	Llave encontrada	Tiempo secuencial	Tiempo paralelo P=4	Speedup
texto6.bin	NO encontrada en 2^24	16.7M llaves en 6.67 s (recorrido total)	—	—
texto7.bin	3412587	23.603 ±0.266	6.469 ±0.039 (P=4 ciclica)	3.649 / 91.2%
textoequipo10.bin	27124	0.0117 ±0.0014	0.00311 (P=4 ciclica)	3.768 / 94.2%

8.5 Ejemplo de 10 mediciones

Tabla llena con la serie texto7 P=4 cíclica (ver Cuadro 4).

Cuadro 4: Diez repeticiones.

Rep.	**Secuencial (s)**	**Paralelo (s)**	**Speedup**	**Eficiencia**
1	23.603	6.541	3.609	90.2 %
2	23.603	6.484	3.640	91.0 %
3	23.603	6.544	3.607	90.2 %
4	23.603	6.447	3.661	91.5 %
5	23.603	6.453	3.658	91.4 %
6	23.603	6.442	3.664	91.6 %
7	23.603	6.443	3.663	91.6 %
8	23.603	6.457	3.655	91.4 %
9	23.603	6.443	3.663	91.6 %
10	23.603	6.437	3.667	91.7 %
Prom.	**23.603**	**6.469**	**3.649**	**91.2 %**

9. Conclusiones

• Versión secuencial y paralela funcionales: se descifraron texto7 (llave 3412587) y textoequipo10 (llave 27124); texto6 no está en [0, 2^24). Validación defensiva 16/16.

• El reparto por bloques es muy sensible a la llave: con la fácil (8388609 = L/2+1) el proceso 1 la halla en ~258 intentos (~0.002 s), un speedup aparente extremo que no refleja mejora real del hardware.

• El reparto cíclico baja de forma estable al agregar procesos (fácil: 23.6 → 12.3 → 8.8 s; media: 29.7 → 15.5 → 11.2 s; difícil: 10.6 → 5.4 → 3.9 s) y en texto7 logró S = 3.65 con P = 4 (E = 91%).

• Se descifraron 2 textos externos (texto7 y equipo10), cumpliendo el mínimo exigido.

10. Recomendaciones

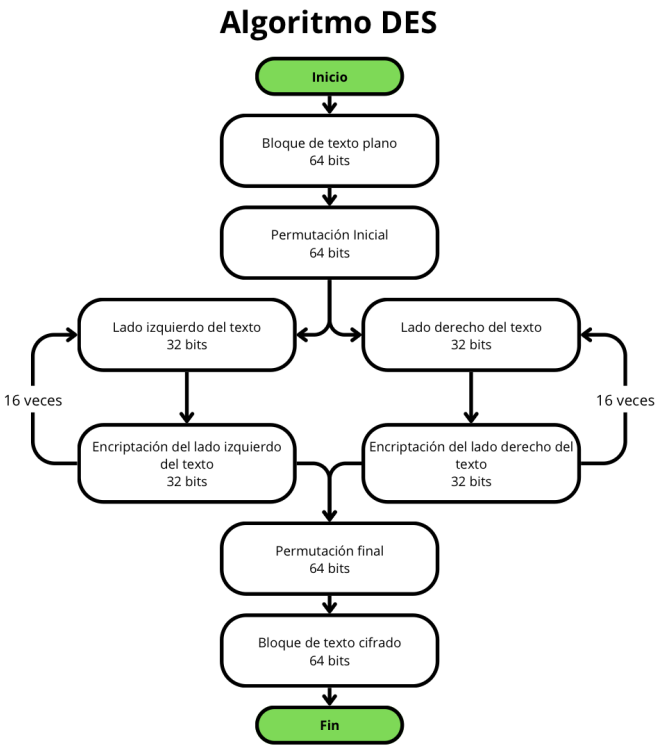
Los principales retos fueron reemplazar la librería DES original, unificar la forma de construir la llave entre máquina. Recomendamos medir siempre en la misma computadora, no mezclar versiones al intercambiar textos entre grupos, usar frases de búsqueda claras y guardar los

archivos CSV originales como evidencia. Como mejora futura se puede probar con más procesos o con varias computadoras conectadas en red.

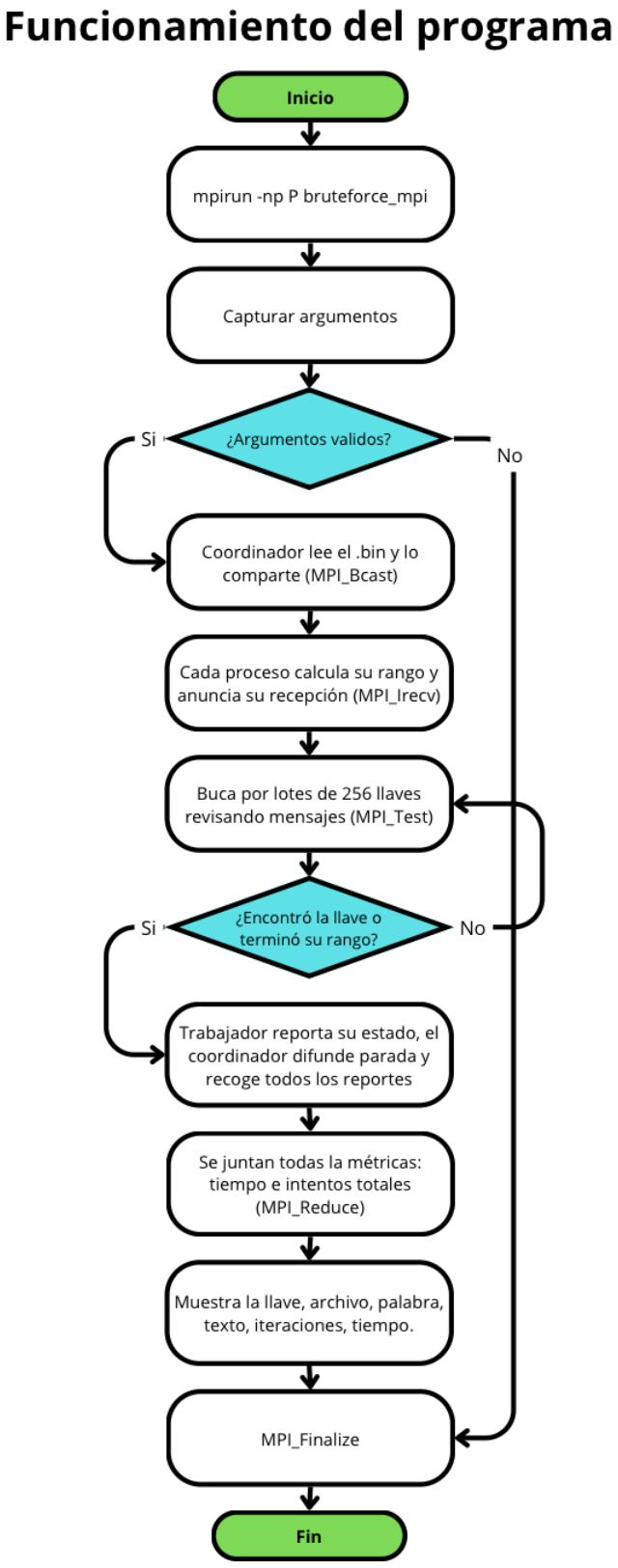
11. Referencias

- [1] NIST. FIPS PUB 46-3: Data Encryption Standard (DES). 1999.
- [2] MPI Forum. MPI: A Message-Passing Interface Standard, v4.0. 2021. https://www.mpi-forum.org/docs/
- [3] OpenSSL Project. Documentación de DES. https://www.openssl.org/docs/
- [4] Foster, I. Designing and Building Parallel Programs. Addison-Wesley, 1995.

A. Diagrama de flujo del algoritmo DES



B. Diagrama de flujo del programa



C. Catálogo de funciones

Resumen de las partes principales del código con sus entradas y salidas.

Cuadro B.1: Funciones principales.

Parte	Qué recibe	Qué entrega	Para qué sirve
Preparar llave	Número de llave	Llave DES de 8 bytes	Convierte el número en formato DES.
Cifrar / Descifrar	Llave y texto	Texto transformado	Cifra o descifra por bloques.
Probar llave	Llave, texto y frase	Sí / no	Prueba si esa llave revela la frase.
Leer archivo	Ruta del archivo	Contenido en memoria	Carga el texto cifrado.
Buscador secuencial	Archivo, frase, límite	Llave + tiempos	Prueba llaves una por una.
Buscador paralelo	Lo mismo + reparto	Llave + tiempos	Reparte la búsqueda con MPI.

Cuadro B.2: Apoyo de MPI

Apoyo MPI	Para qué sirve
Inicio / fin de MPI	Abrir y cerrar el entorno paralelo.
Averiguar procesos	Saber cuántos somos y quién soy.
Compartir datos	Enviar el texto cifrado a todos.
Avisos sin espera	Reportar hallazgos sin frenar la búsqueda.
Juntar resultados	Quedarse con el tiempo mayor y sumar intentos.

Cuadro B.3: Scripts de medición.

Script	Qué hace
medir_secuencial.sh	Repite la búsqueda secuencial y promedia.

medir_paralelo.sh	Repite la paralela por procesos y reparto.
probar_mpi_defensivo.sh	Verifica que el programa maneja errores y no se traba.

D. Bitácora de pruebas

Cada medición se repite 10 veces y se guarda en resultados/secuencial.csv y resultados/paralelo.csv. Los comandos usados están en la sección 8.

Cuadro C.1: Ejemplo real de filas del CSV (texto7, 13/10/2026).

fecha	corrida	archivo	llave	tiempo_s	limite	iteraciones
2026-10-13T14:44:06	1	data/texto.bin	8675309	45.512474	16777216	8675310
2026-10-13T11:08:02-06:00	2	data/texto.bin	8675309	45.616414	16777216	8675310
2026-10-13T11:08:48-06:00	3	data/texto.bin	8675309	45.523417	16777216	8675310